Ders 6 Genişletilmiş Notu: Seminer Sunumları: CNN Sıkıştırma, DeepZip, v2vDC, FastText.zip, IDBE, BERT ile Metin Sıkıştırma ve Öğrenilmiş Görüntü Kod Çözücüleri

Ders 6 Genişletilmiş Notu: Seminer Sunumları: CNN Sıkıştırma, DeepZip, v2vDC, FastText.zip, IDBE, BERT ile Metin Sıkıştırma ve Öğrenilmiş Görüntü Kod Çözücüleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Seminer Dersinin Düzeni
- 2. Evrişimsel Sinir Ağlarının Sıkıştırılması
- 3. DeepZip: RNN ile Kayıpsız Sıkıştırma
- 4. Dinamik v2vDC: Kelime Tabanlı Sözlük Sıkıştırıcı
- 5. FastText.zip: Metin Sınıflandırma Modellerinin Sıkıştırılması
- 6. IDBE: Akıllı Sözlük Tabanlı Kodlama
- 7. BERT Tabanlı Kısa Metin Sıkıştırma
- 8. Kayıplı Görüntü Sıkıştırmada Öğrenilmiş Yinelemeli Kod Çözücü
- 9. RNN ile Tam Çözünürlüklü Görüntü Sıkıştırma
- Örnek Sorular
- Dersin Toparlaması

1. Seminer Dersinin Düzeni

Bu ders, bir önceki dersteki gibi öğrenci seminerlerine ayrıldı. Her öğrenci veri sıkıştırmayla ilgili yayımlanmış bir makaleyi en çok 15 dakikada anlattı. Hoca her sunumun sonunda makalenin hangi yıla ait olduğunu sordu, sınıfa soru olup olmadığını sordu ve kendi sorularını yöneltti. Hoca son yıllara ait makalelerin seçilmesini önemsedi.

Toplam sekiz sunum yapıldı. Aşağıda her sunum ayrı bir bölüm olarak, anlatıldığı sırayla verilmiştir. Sunanlar “bir öğrenci” diye anılmıştır. Sunumların çoğu sinir ağı tabanlı sıkıştırmayla ilgilidir: ya bir sinir ağının kendisi sıkıştırılır (CNN, FastText) ya da bir sinir ağı sıkıştırma için tahminci olarak kullanılır (DeepZip, BERT, öğrenilmiş kod çözücüler).

2. Evrişimsel Sinir Ağlarının Sıkıştırılması

2.1 CNN ile yapay sinir ağının farkı

İlk sunumu yapan öğrenci, doğal dil işlemede kullanılan evrişimsel sinir ağlarının (CNN) sıkıştırılmasını anlatan bir makaleyi sundu; öğrenciye göre makale 2018 tarihlidir. Öğrenci önce CNN’in klasik yapay sinir ağından (YSA) farkını açıkladı. Aslında ikisi birlikte kullanılır: girdi verisi, YSA’ya girmeden önce evrişim (convolution) ve ortaklama (pooling) katmanlarından geçer. Bunun amacı girdinin boyutunu küçültmek, YSA’ya daha az veri göndermek ve hesaplamayı hafifletmektir.

CNN ilk olarak bilgisayarla görmede kullanıldı, çünkü bir resim zaten bir matristir (gri tonlu, siyah-beyaz ya da renkli; üçü de matristir). Evrişim işleminde filtre, girdi matrisinin üzerinde gezdirilir; filtrenin kapladığı her konumda karşılıklı pikseller çarpılır ve çarpımlar toplanır. Sonuçta girdiden daha küçük bir matris elde edilir. Gerçekte birden fazla filtre kullanılır.

Ortaklama katmanında filtrenin yalnız boyutu vardır, ağırlığı yoktur. Örneğin 2×2'lik bir pencere evrişim çıktısının üzerinde gezdirilir (ekrandan, 00:10:00):

- Maksimum ortaklama: penceredeki dört değerin en büyüğü çıktıya yazılır.
- Ortalama ortaklama: penceredeki değerler toplanıp dörde bölünür.

2.2 Doğal dil işlemede CNN

CNN genelde sınıflandırma problemlerinde kullanılır; doğal dil işlemede de önce metin sınıflandırma ve duygu analizinde kullanılmaya başlandı. Bir cümle bir matrisle ifade edilir. Öğrenci "can I eat the pizza" cümlesini one-hot kodlamayla gösterdi: sözlük (vocabulary) genellikle alfabe sırasıyla kurulur; cümledeki her kelime için matrisin bir satırı ayrılır ve kelimenin sözlükteki sırasına karşılık gelen sütun 1, geri kalanlar 0 olur. Bir metin, arka arkaya gelen bu matrislerden oluşur. Evrişim ve maksimum ortaklamadan sonra daha küçük matrisler elde edilir, düzleştirme (flatten) ile YSA'ya verilir. Filtre sayısı arttıkça çıkan matris sayısı da artar, ama matrisler küçük olduğundan YSA'ya yine az veri gider.

2.3 Sıkıştırmanın avantajları ve dezavantajı

CNN'lerin sıkıştırılmasının iki temel avantajı vardır: hesaplama yükü azalır ve bellek daha verimli kullanılır. Dezavantajı, eşik ve parametreler doğru seçilmezse modelin doğruluğunun (accuracy) büyük ölçüde zarar görmesidir. Öğrenci üç temel yöntemi anlattı: budama (pruning), nicemleme (quantization) ve Huffman kodlaması.

2.4 Budama (pruning)

Budama, ağdan ağırlıkların (bağlantıların), düğümlerin ya da evrişim katmanındaki filtrelerin silinmesidir.

- Bağlantı budama: bir eşik değeri belirlenir, ağırlığı bu eşiğin altında kalan bağlantılar silinir. Ağ mimarisi değişmez, yalnız karmaşıklık azalır. Öğrencinin makalesinde eşik 0,02 seçilmiş; ağırlıklar histograma dökülüp eşik altındakiler ayrılmış.
- Düğüm budama: çıktısı genellikle sıfıra yakın olan ve seyrek olarak yüksek değerle etkinleşen düğüm silinir. Karmaşıklığı önemli ölçüde azaltır, ama yanlış seçilirse doğruluğu ciddi biçimde bozar.
- Filtre budama: evrişim katmanındaki filtreler silinir.

Budama düzeyi (düğüm, bağlantı, filtre, kanal) ve budama ölçütü (eşiğin nasıl belirleneceği) ayrı ayrı seçilir. Makalede filtre budamasıyla doğruluğun düşmediği, hatta mikro ve makro değerlerde arttığı gösterilmiş.

2.5 Nicemleme (quantization)

Nicemlemenin asıl amacı hesaplama yükünden çok belleği verimli kullanmaktır. Sürekli değişen (float) veriler kesikli tamsayılara çevrilir. m bitlik bir tamsayı 2^m farklı değer gösterebilir; float tipi çok daha geniş bir aralığı gösterir ve bellekte daha çok yer tutar. Tamsayıya çevirmek bellek kazandırır.

Slaytta FP32'den INT8'e dönüşüm formülleri verildi (ekrandan, 00:18:40). f_{min} ve f_{max} girdi verisinin sınırları, q_{min} ve q_{max} nicemlenmiş verinin sınırlarıdır:

$$Q(x) = \text{floor}\left(\frac{x}{\text{scale factor}} + \text{zero point}\right)$$

$$\text{scale factor} = \frac{f_{max} - f_{min}}{q_{max} - q_{min}}, \qquad \text{zero point} = \frac{f_{min} + f_{max}}{2}$$

Ders sonrası not: Slayttaki INT8 ekseninin uçları −127 ve −128 diye yazılmıştı; INT8 aralığı büyük olasılıkla −128 ile 127 olmalıdır. Slayttaki sıfır noktası formülü de yaygın kullanılan zero point $= q_{min} - f_{min}/\text{scale}$ tanımından farklıdır; bu formül büyük olasılıkla slayta yanlış aktarılmıştır.

Nicemlemenin iki türü vardır:

- Eğitim sonrası nicemleme (post-training quantization): eğitim tamamen bittikten sonra sürekli değerler kesikli değerlere çevrilir.
- Nicemlemeye duyarlı eğitim (quantization aware training): model her geri yayılım (backpropagation) yaptığında yeniden nicemleme yapılır.

TensorFlow'un sitesinden alınan bir tabloda, nicemlemeye duyarlı eğitimde doğruluğun çok az düştüğü görüldü, çünkü her geri yayılımdan sonra yeniden nicemleme yapılır. BERT ile CNN'in birlikte kullanıldığı bir tabloda da nicemleme sonrası doğruluk kaybının küçük olduğu gösterildi. Makaledeki üç boyutlu grafik budama eşiği ile nicemleme bit sayısının (32, 16, 8 bit) birlikte doğruluğu nasıl etkilediğini gösteriyordu.

2.6 Huffman kodlaması ve üç adımın birleşimi

Huffman nicemlemeyle birlikte kullanılır: ağırlıklar tamsayıya çevrildikten sonra bu kesikli değerleri kodlamak kolaylaşır ve Huffman ile üçüncü kez bellek kazanılır. Üç yöntem genellikle şu sırayla birlikte uygulanır (ekrandan, 00:20:00):

```
özgün ağ
  -> Budama (daha az ağırlık): bağlantıyı eğit, bağlantıları buda, ağırlıkları eğit (döngü)
     => aynı doğruluk, 9x-13x küçülme
  -> Nicemleme (ağırlık başına daha az bit): ağırlıkları kümele, kod kitabı üret,
     ağırlıkları kod kitabıyla nicemle, kod kitabını yeniden eğit (döngü)
     => aynı doğruluk, 27x-31x küçülme
  -> Huffman kodlama: ağırlıkları kodla, indeksi kodla
     => aynı doğruluk, 35x-49x küçülme
```

Ek bilgi (derste anlatılmadı): Bu üç aşamalı şema, büyük olasılıkla Han, Mao ve Dally'nin "Deep Compression" (2016) çalışmasından alınmıştır. Orada nicemleme, ağırlıkların k-means ile kümelenip her ağırlığın yalnız küme indeksiyle saklanması biçiminde yapılır; Huffman da bu indekslerin dengesiz dağılımından yararlanır.

3. DeepZip: RNN ile Kayıpsız Sıkıştırma

3.1 RNN, LSTM ve GRU

İkinci sunumu yapan öğrenci, yinelemeli sinir ağlarıyla (RNN) kayıpsız sıkıştırma yapan DeepZip makalesini anlattı. Öğrenci önce üç kavramı tanıttı.

- RNN: düğümler arasındaki bağlantıların yönlendirilmiş bir döngü oluşturduğu sinir ağı sınıfıdır. Geçmişteki verilerden yararlanarak yeni veriyi tahmin eder. Geri yayılım ilerledikçe ağırlıkların gradyanı katlanarak küçülür ve kaybolur; bir süre sonra geçmiş verinin bilgisine ulaşılamaz.
- LSTM (uzun kısa süreli bellek): temel kavramları hücre durumu ve kapılardır. Hücre durumu, anlamlı bilgiyi hücreler boyunca taşıyan bir iletişim hattı, yani ağın hafızasıdır. Kapılar hangi bilginin gerekli olduğunu belirler; sigmoid fonksiyonuyla değerleri 0–1 aralığına sıkıştırır. 0 olan bilgi unutulur, 1 olan bilgi hücre durumunda ilerler.
- GRU: hücre durumu yoktur, bilgi aktarmak için gizli durumu kullanır. Yalnız iki kapısı vardır: güncelleme kapısı (LSTM'in unutma ve girdi kapılarına benzer şekilde hangi bilginin atılacağına ve ekleneceğine karar verir) ve sıfırlama kapısı (ne kadar geçmiş bilginin unutulacağına karar verir).

3.2 DeepZip'in yapısı

DeepZip, RNN tabanlı bir olasılık tahmincisi ile bir aritmetik kodlayıcıdan oluşur. Tahminci bloğu GRU ya da başka bir RNN olabilir; sonunda bir softmax katmanı bir sonraki sembolün olasılık dağılımını verir. Bu dağılım aritmetik kodlayıcıya beslenir. Aritmetik kodlayıcı her

sembol dizisi için 0–1 aralığında benzersiz bir alt aralık bulur; aralık her adımda bir sonraki sembolün tahmin olasılığına göre daralır. Sonuçta veri bu aralıktan bir sayı olarak sıkıştırılır. Kod çözme aynı olasılık tahminleriyle ters yönde yapılır.

Kodlayıcı ilk sembol S_0 için varsayılan bir olasılık dağılımıyla başlar; böylece kod çözücü ilk sembolü de çözebilir. Hem aritmetik kodlayıcı hem tahminci, yinelemeler arasında durum bilgisini aktarır. Model çok sayıda epoch eğitilmediyse tahmincinin ağırlıkları da ayrıca saklanmalıdır ve bu boyut sıkıştırılmış boyuta eklenir.

3.3 Deneyler

Çalışmada entropisi bilinen yapay veri setleri ve gerçek veriler kullanıldı. Modeller:

- DeepZip RNN: karakter düzeyinde, sade RNN tabanlı model.
- DeepZip ChGRU: karakter düzeyinde GRU tabanlı model.
- Üçüncü model: yalnız önceki sembole değil, önceden gözlenen bütün sembollere bağlı GRU tabanlı model.

Sonuçlar:

- IID kaynaklarda en küçük sade RNN bile kolayca optimize eder, çünkü modelin belleğe ihtiyacı yoktur. 16 hücre boyutlu DeepZip ChGRU birkaç yinelemede entropi sınırına ulaşır (ekrandan, 00:30:00).
- Markov-k kaynaklarında k büyüdükçe sade RNN'de kaybolan gradyan sorunu başlar; GRU, RNN'den daha büyük k değerlerinde sıkıştırabilir.
- Geri yayılımdaki dizi uzunluğunun etkisi: dizi uzunluğu 8 ile eğitilen GRU, dizi uzunluğundan daha uzun bağımlılıkları öğrenebilse de, dizi uzunluğunun yaklaşık iki katından sonra zorlanır.
- Sözde rastgele sayı üreteci (PRNG) dizilerinde Gzip kötü performans gösterdi; bu diziler neredeyse rastgele olduğundan beklenmedik değildir. DeepZip modelleri ise bu dizileri önemli ölçüde sıkıştırabildi, yani dizinin aslında rastgele olmadığını yakaladı.
- DNA verisinde DeepZip, özel DNA sıkıştırıcısı MFCompress'ten daha iyi sonuç verdi.
- Metin verisinde (Wikipedia tabanlı veri setleri) DeepZip daha iyi performans gösteremedi, çünkü metinde çok uzun bağımlılıklar yoktur.

Sonuç olarak makale, dizilerin kayıpsız sıkıştırılması için sinir ağı tahminine dayalı bir çerçeve önermiş; Gzip'e göre iyileşme sağlamış ve yapay verilerde entropi sınırına yakın sıkıştırmaya ulaşmıştır.

4. Dinamik v2vDC: Kelime Tabanlı Sözlük Sıkıştırıcı

4.1 Arka plan

Üçüncü sunumu yapan öğrenci, 2019 tarihli dinamik bir kelime tabanlı sözlük sıkıştırıcıyı anlattı. Bu tür sıkıştırıcılar, farklı uzunluktaki girdi sembollerini toplar ve sık kullanılan sembollere kısa kod kelimeleri veren bir sözlük oluşturur. Öğrenci mevcut yöntemlerin gelişimini şöyle özetledi: Huffman kodlaması 1950'lerde tanıtıldı; karakter tabanlıdır ve sıkıştırma oranı düşüktür. Daha sonra kelime tabanlı modeller denendi; 90'larda kod çözmeyi hızlandıran geliştirmeler yapıldı. Burada yalnız sıkıştırma oranı değil, açma (decompress) hızı ve kayıpsızlık da ölçüttür. Ancak bu tekniklerin hiçbiri p7zip kadar güçlü sıkıştıramıyordu.

Statik v2vDC metni kelime ve kelime dizilerine ayırıp (parse) her birine bir kod kelimesi atar; özgün kelimeler bu kodlarla değiştirilerek sıkıştırılmış veri oluşur. Sıkıştırma oranı yaklaşık %10 iyileşir ve sıkıştırılmış metin üzerinde arama yapılabilir (ekrandan, 00:40:00).

Soru: Sıkıştırılmış veri üzerinde mi arama yapılıyor? **Cevap:** Evet. Sık kullanılan kelimeler ve kelime dizileri kod kelimeleriyle değiştirildiğinden, sıkıştırılmış metin de kod kelimelerinden oluşan anlamlı bir metin olarak kalır; aranan kelimenin kodu bulunup sıkıştırılmış metinde aranabilir ve özgün metne geri dönülebilir.

4.2 Dinamik sürüm

Veri iletiminde metnin tamamını beklemek mümkün değildir; bu yüzden dinamik bir yöntem gerekir. Dinamik sürümde her kelimeden sonra sözlük güncellenir; kullanım sıklığına göre yeni sıralama belirlenir ve kod kelimeleri hemen değişebilir. Gönderici ve alıcı eşzamanlı çalışır: bir kelime geldikçe sıkıştırılmış hali karşı tarafa iletilir. Semboller bir kelime ya da bir kelime dizisi olabilir.

Çalışma üç ana aşamadan oluşur: gönderici metni ayrıştırır (parse), sembolleri kodlar, alıcı aynı işlemleri simetrik olarak yapar.

4.3 Ayrıştırma: terminal ve non-terminal semboller

Slayttaki tanımlar (ekrandan, 00:43:40):

- Terminal sembol: yeni bir kelime ayrıştırıldığında üretilir.
- Non-terminal sembol: iki farklı sembolün birleşimidir, birden fazla kelimeyi temsil eder.
- Okunan kelime sözlükte yoksa iki sembol üretilir: yeni kelimeyi temsil eden terminal sembol ve az önce üretilen sembolü bir önceki sembole ekleyen $S{+}1$'inci non-terminal sembol (ikinci kelime için $S_1 \mid S_{new}$).

Örnek metin "the more I know about you" için oluşan semboller:

```
S0: "the"        S1: "more"        S2: "the more"
S3: "I"          S4: "more I"      S5: "know"
S6: "I know"     S7: "about"       S8: "know about"
S9: "you"        S10: "about you"
```

Slayttaki tabloda her sembolün sol (l) ve sağ (r) bileşeni, sıklığı (freq) ve bir "voc" sütunu vardır (ekrandan, 00:43:40):

	l	r	freq	voc
0	the	-1	1	0
1	more	-1	1	1
2	0	1	0	9
3	I	-1	1	2
4	1	3	0	7
5	know	-1	1	3
6	3	5	0	6
7	about	-1	1	4
8	5	7	0	8
9	you	-1	1	5
10	7	9	0	10

Terminal sembollerde r = -1'dir; non-terminal sembollerde l ve r birleştirilen iki sembolün indisidir (örneğin 2 numaralı sembol 0 ve 1'in, yani "the" ve "more"un birleşimidir). Sembol listesi python ile yeniden üretildi ve slayttaki listeyle aynı çıktı.

Cümlenin devamı "the more I know about you, the more I know about myself" olduğunda, ikinci kez "the more" geldiğinde bu sembolün kullanım sıklığı artırılır. Böylece yeni gelen kelimelere göre sıklıklar sürekli güncellenir.

Soru: (Hoca tabloyu adım adım okuttu.) "the" ve "more" eklendikten sonra neden bir boşluk bırakılıyor? **Cevap:** Boşluk, alıcıya iki sembolün birleşiminden yeni bir sembol eklendiğini bildirmek içindir. Alıcı bu boşluğu görünce kendi tarafında da birleşik sembolü oluşturur. Hoca bu noktayı farklı anladığını, boşluğun yalnız karşı tarafa sembol eklendiğini bildirdiğini söyleyerek netleştirdi.

4.4 Kodlama ve sıklık takibi

Her sembol kodlanıp alıcıya gönderilir. Kodlamada yoğun bir önek kod kullanıldığı söylendi, ama makale ayrıntı vermeyip kaynak göstermiştir.

Kullanım sıklığı için gönderilen her sembolün kaydı tutulur; iki liste vardır: biri sembolün konumunu, diğeri kullanım sıklığını tutar. Her yeni sembolde sıklık bir artırılır ve listenin hâlâ

güncel olup olmadığına bakılır. Sıralama değişirse kod kelimelerinin sırası da değişir; sık kullanılan sembol daha kısa kodla ifade edilir.

Alıcı tarafı göndericiyle simetrik çalışır: bir sembol çözüldükten sonra göndericinin yaptığı gibi yeni birleşik sembolü oluşturur. Metin bittikten sonra değil, metinle eşzamanlı çalışır; dinamik olmanın özelliği budur.

> **Ek bilgi (derste anlatılmadı):** End-Tagged Dense Code (ETDC), kelimeleri sıklık sırasına göre bayt dizileriyle kodlayan bir yöntemdir. Her kod kelimesinin son baytında bir bit işaret olarak kullanılır, böylece kod kelimesinin bittiği yer önek kod gibi belirlenir. Bayt hizalı olduğu için sıkıştırılmış metin üzerinde doğrudan arama yapmaya uygundur.

4.5 Sonuçlar

Slayttaki tabloda sıkıştırılmış boyutun düz metin boyutuna oranı (%) verildi; küçük değer daha iyi sıkıştırmadır (ekrandan, 00:46:40). Seçilmiş satırlar:

Veri	Detdc	v2vdc (lzma)	v2vdc (plain)	D-v2v	Repair +sHuff	lzma def	lzma -9 -e	p7zip	bzip2 def	Boyut (KB)
FT91	35,64	27,15	30,11	28,60	24,00	25,50	25,25	25,52	27,06	14.404
CR	31,99	23,55	24,73	22,86	20,16	22,05	20,83	21,63	24,14	49.888
ZIFF	33,79	24,01	24,66	23,14	20,33	23,40	21,64	22,98	25,10	180.879
ALL	33,66	22,81	23,39	22,67	-	23,23	21,34	22,80	25,98	1.055.391

Öğrenci sonucu şöyle açıkladı: %23 oranı, 100 MB'lik verinin 23 MB'ye düştüğü anlamına gelir. Önerilen dinamik yöntem (D-v2v) en iyi sıkıştırıcı p7zip'e çok yakın sonuç verdi. Asıl avantajı açma tarafındadır: p7zip iyi sıkıştırır ama yavaş açar, dinamik iletimde sorun çıkarır; dinamik yöntem açmada p7zip'in önüne geçer. En büyük dezavantajı, her iki kelimeden yeni bir sembol ürettiği için çok bellek gerektirmesidir. Makale gelecek çalışmaların bellek gereksinimini azaltmaya yöneleceğini belirtmiştir.

5. FastText.zip: Metin Sınıflandırma Modellerinin Sıkıştırılması

5.1 Arka plan

Dördüncü sunumu yapan öğrenci, Facebook AI araştırma ekibinin metin sınıflandırma modellerini sıkıştırmak için yaptığı FastText.zip çalışmasını anlattı. Öğrenci makalenin 2017'de bir konferansta yayımlandığını söyledi. Aynı ekip FastText'i de geliştirmiştir. FastText metin sınıflandırma ve kelime gösterimlerini öğrenmek için kullanılır; özellikle sözlükte olmayan kelimelerin vektörlerini öğrenebilir. Word2Vec'ten farkı, kelimeleri n-gramlara bölüp sinir ağına bu şekilde vermesidir. Örneğin "elma" kelimesi trigramlara bölünürse "elm" ve "lma" vektörleri oluşur.

Her kelime için ayrı n-gramlar oluşturulduğundan n-gram sayısı çok büyüktür; eğitim uzun sürer ve modeller büyür. Bu, alanda çalışmayı zorlaştırır. Ekip bu yüzden nicemleme tabanlı bir sıkıştırma yöntemi önermiştir. Sonuç modeli FastText'ten çok daha az yer kaplar, ama sert bir budama ve nicemleme yapıldığı için performansı biraz düşüktür. Ekibin amacı doğruluğu sabit tutmak değil, olabildiğince az düşürerek boyutu küçültmektir; FastText'in sınıflandırma başarısını zaten yeterli bulmuşlardır.

5.2 İlgili çalışmalar

- Budama (pruning): bir eşik belirlenir, eşiğin altında olasılığa sahip n-gramlar dışarıda bırakılır ve en önemli n-gramlar modelde tutulur.
- Softmax yaklaşımı: diğer çalışmalarda n-gramların benzerliği ya da önemi Öklid uzaklığı veya kosinüs benzerliği ile ölçülmüş; FastText'te softmax katmanı kullanılmıştır. Softmax girdileri olasılık değerlerine çevirir.
- CNN'ler için yapılan sıkıştırma çalışmaları (nicemleme, ikilileştirme) yüksek verim göstermiştir; ekip bunları rakip olarak görmüştür. Bu çalışmalar hem model boyutunu küçültmeyi hem performansı artırmayı amaçlar.

5.3 Ürün nicemleme (PQ)

Önerilen yöntem ürün nicemleme (product quantization, PQ) olarak adlandırılmıştır. Nicemleme, gerçek değerleri önceden tanımlı bir kümedeki en yakın vektöre yaklaştırmaktır. Alt nicemleyiciler az sayıda merkeze sahip olduğu için eğitim çok hızlıdır; öğrenciye göre genellikle iki merkez vektörü kullanılmış ve her gerçek değer hangisine yakınsa ona yaklaştırılmıştır.

İkilileştirme (binarization) ile nicemleme karşılaştırıldı. İkilileştirmede bir eşik bulunur; değer eşikten büyükse 1, küçükse 0 olur. Nicemlemede birden fazla seviye belirlenir ve aralıklara düşen değerler bu seviyelere atanır. Slayttaki resim örneğinde sağdaki nicemleme üç seviye kullanıyordu. Nicemlemenin ikilileştirmeden daha iyi performans verdiği söylendi. Nicemlemeden sonra yeniden eğitim (retraining) de yapılmıştır; önceki çalışmalarda bunun verimi artırdığı görülmüştür.

5.4 Budama, hashing ve Bloom filtresi

Bellek kullanımı sözlüğün büyüklüğüne bağlıdır, bu yüzden budama da gerekir. Kelimelerin normlarına bakılmış: “the”, “and”, “to” gibi çok sık kullanılan kelimeler ayırt edici değildir, anlam katmaz; bunları çıkarmak doğruluğu çok etkilemez. Makale budamanın ne kadar sert yapıldığını ayrıntılı vermemiştir; “aşırı budama” (extreme pruning) yapıldığı söylenmiştir.

Hashing ve Bloom filtresi slaytta şöyle karşılaştırıldı (ekrandan, 01:00:00):

- HashMap ile gelen her farklı değerin sözlükte olup olmadığına bakılır. İşlem karmaşıklığı açısından $O(1)$ basit bir problemdir. Ancak saniyede birkaç milyon olayın işlendiği yoğun bir uygulamada bellek kullanımı aşırı artar.
- Bloom filtresi, birden fazla hash değerinin sonuçlarını bitler halinde kaydeden bir bit dizisidir. Test zamanında $O(1)$ karmaşıklığı korur ve birkaç yüz kilobayt tasarruf sağlar, ancak pratikte performansı düşürür.

Bloom filtresi bir değerin daha önce geçmediğini kesin söyleyebilir, ama geçtiğini %100 doğrulukla söyleyemez. Bu yüzden çalışmada Bloom filtresi kullanılmamış, hashing ile devam edilmiştir.

5.5 Deneyler ve sonuçlar

Varsayılan ayarlarla 8 veri setinde modeller eğitilmiş; her veri setinde birkaç milyon belge ve en çok 10 sınıf vardır. Önce budama, sonra nicemleme yapılmıştır. Öğrenci “10 kat sıkıştırma” ifadesini 100 MB’nin 10 MB’ye inmesi diye açıkladı.

Çok küçük modellerdeki doğruluk tablosu (ekrandan, 01:01:35):

Veri seti	Boyut	full	64KiB	32KiB	16KiB
AG	65M	92.1	91.4	90.6	89.1
Amazon full	108M	60.0	58.8	56.0	52.9
Amazon pol.	113M	94.5	93.3	92.1	89.3
DBPedia	87M	98.4	98.2	98.1	97.4
Sogou	73M	96.4	96.4	96.3	95.5
Yahoo	122M	72.1	70.0	69.0	69.2
Yelp full	78M	63.8	63.2	62.4	58.7
Yelp pol.	77M	95.7	95.3	94.9	93.2
Ort. fark [%]		0	-0.8	-1.7	-3.5

64 KiB’ye kadar sıkıştırıldığında doğruluk ortalama %0,8, 32 KiB’de %1,7 azalmıştır; slayttaki açıklamaya göre bu, ×1.000–4.000 sıkıştırma oranına karşın elde edilmiştir. 16 KiB’de düşüş daha büyüktür; yazarlar 64 KiB’yi yeterli bulmuştur. Ortalama farklar tablodan python ile yeniden hesaplandı: $-0{,}80$, $-1{,}70$ ve $-3{,}46$ (yuvarlanınca $-3{,}5$).

Gelecek çalışma olarak, girdi özellikleri ve etiketlerin frekansa göre yeniden değerlendirilmesi ve entropi ile budama ölçütlerinin birleştirilmesi önerilmiş. Öğrenci, yazarların kodu FastText’in GitHub hesabında paylaşacaklarını söyledikleri halde paylaşmadıklarını ve bu konuda sonraki bir çalışma yapmadıklarını gördüğünü söyledi.

Soru: Gelecek çalışmaların yapılıp yapılmadığını nasıl kontrol ettin; yazarlara mı baktın, kodun yüklenip yüklenmediğine mi? **Cevap:** Hem kod deposuna hem yazarların sonraki çalışmalarına bakılmış; kullanıcıların kodun ne zaman yükleneceğini sorduğu, yanıt alınmadığı görülmüş. Hoca bunun kesin olmadığını, "emin olmamakla birlikte" diye söylenmesi gerektiğini belirtti.

> **Ders sonrası not:** Öğrenci makalenin yılını önce 2017, sunumun sonunda 2016 olarak söyledi. Çalışma büyük olasılıkla 2016 sonunda ön baskı olarak yayımlanmış, sonra 2017'de konferansa gönderilmiştir.

6. IDBE: Akıllı Sözlük Tabanlı Kodlama

6.1 Amaç ve esin kaynağı

Beşinci sunumu yapan öğrenci, "An Intelligent Dictionary Based Encoding Algorithm" adlı, metin sıkıştırmaya yönelik sözlük tabanlı bir kodlama algoritmasını anlattı. Bütün sıkıştırma algoritmalarının temel amacı fazlalığı (redundancy) azaltmaktır. Verilerin büyümesiyle bant genişliğini verimli kullanmak gerekir; yazar gönderilen verilerin yaklaşık %45'inin metin olduğunu belirtmiştir.

Yazarlar, daha önce başarılı olmuş Burrows-Wheeler dönüşümünden (BWT) esinlenmiştir. BWT gibi metni önişleme ile bir ara forma dönüştürüp, sırasını değiştirerek daha iyi sıkıştırılabilir hale getirmek isterler.

6.2 Benzer çalışmalar

Slayttaki benzer çalışmalar (ekrandan, 01:20:00):

- BWT tek seferde tek bayt ya da birkaç bayt dönüştürebilir. Tipik zincir: `BWT < input-file | MTF | RLE | ARI > output-file`. BWT ve MTF (move-to-front) yalnız dönüşümdür, kodlama yapmaz; sıkıştırmayı RLE ve aritmetik kodlama yapar. Öğrenci MTF'nin derste gösterildiğini hatırlattı.
- IDBE ile tek seferde daha büyük parçaların (large chunks) dönüştürülmesi hedeflenir.
- Star encoding: en sık kullanılan kelimelerden oluşan sözlük tabanlı bir algoritmadır. Sözlük önceden hazırlanmalı, sıkıştırıcı ve açıcı bu sözlüğü bilmelidir. Sözlükteki her kelimenin bir yıldız karşılığı vardır; örneğin "the" kelimesinin karşılığı `t**` olabilir.

6.3 Algoritmanın fikri

Algoritmanın hedefi yüksek sıkıştırma oranlı, iyi dönüşüm yapan bir yöntem tasarlamaktır. Makalede şu örnek verilir: "The objective of this paper is develop a better transformation" cümlesinde bazı harfler eksik yazılsa bile göz doğrusunu ayırt edebilir; çünkü beyin görsel hafızasında hatalı metinlere karşılık gelen doğru metinlerin bir sözlüğünü tutar. Yazarlar bu fikri benimseyerek önceden tasarlanmış bir sözlükle dönüşüm yapmayı önermiştir.

Soru: Sözlük harfi eksik kelimelerden mi oluşturulmuş? **Cevap:** Tam olarak öyle değil; makale yalnız bu örneği vermiştir. Hoca algoritmada eksik karakterle ilgili bir adım göremediğini belirtti. Öğrenci, örneğin yalnız sözlük fikrini anlatmak için verildiğini, algoritmanın en yakın eşleşmeyi aramadığını; kelime sözlükte birebir varsa işlem yaptığını, yoksa olduğu gibi yazdığını açıkladı.

6.4 Algoritma

Algoritma iki adımdan oluşur: önce sözlüğü oluştur, sonra girdiyi kodla. Slayttaki sözde kod (ekrandan, 01:23:40):

```
Start encode with argument input file inp
A. Read the dictionary and store all words and their codes in a table
B. While inp is not empty
   1. Read the characters from inp and form tokens.
   2. If the token is longer than 1 character, then
      1. Search for the token in the table
```

```
      2. If it is not found,
            1. Write the token as such in to the output file.
         Else
            1. Find the length of the code for the word.
            2. The actual code consists of the length concatenated with the code
               in the table, the length serves as a marker while decoding and is
               represented by the ASCII characters 251 to 254 with 251
               representing a code of length 1, 252 length 2 and so on.
      3. Write the actual code into the output file.
      4. Read the next character and neglect it if it is a space. If it is any
         other character, make it the first character of the next token and go
         back to B, after inserting a marker character (ASCII 255) to indicate
         the absence of a space.
      Endif
   Else
      1. Write the 1 character token
      2. If the character is one of the ASCII characters 251-255, write the
         character once more so as to show that it is part of the text and
         not a marker
   Endif
End (While)
C. Stop.
```

Özetle: tablodaki bir kelime “uzunluk işaretçisi + kod” olarak yazılır; uzunluk işaretçisi ASCII 251–254 arasındadır. Kelimeden sonraki boşluk yazılmaz; boşluk yoksa ASCII 255 işaretçisi eklenir. Metnin kendisinde 251–255 karakterleri geçerse, işaretçiyle karışmasın diye iki kez yazılır.

Soru: (Hoca sözde kodu büyüttürüp tekrar inceledi.) Bulunamazsa token olduğu gibi yazılıyor, bulunursa kodun uzunluğu bulunuyor. Peki tabloya ne konuyor, kelimenin yanlışı mı doğrusu mu? **Cevap:** Makale tabloyu ayrıntılı açıklamamıştır; 8 sayfalık bir bildiri olduğu için ayrıntı yoktur. Hoca sempozyum bildirilerinde sayfa sınırı olduğunu, ayrıntının belki yazarların kendi sayfasında bulunabileceğini söyledi.

6.5 Performans analizi

Üç yöntem karşılaştırıldı: yalnız BWT, BWT + star encoding (*Encode) ve BWT + IDBE. Ölçütler karakter başına bit (BPC) ve dönüşüm süresidir. Calgary korpusu sonuçları (ekrandan, 01:26:35):

Dosya	Boyut(Kb)	BWT BPC	Süre	*Encode BPC	Süre	IDBE BPC	Süre
bib	108.7	2.11	1	1.93	6	1.69	4
book1	750.8	2.85	11	2.74	18	2.36	11
book2	596.5	2.43	9	2.33	14	2.02	10
geo	100.0	4.84	2	4.84	6	5.18	5
news	368.3	2.83	6	2.65	10	2.37	7
paper1	51.9	2.65	1	1.59	5	2.26	3
paper2	80.3	2.61	2	2.45	5	2.14	4
paper3	45.4	2.91	2	2.60	6	2.27	3
Paper4	13.0	3.32	2	2.79	5	2.52	3
Paper5	11.7	3.41	1	3.00	4	2.8	2
Paper6	37.2	2.73	1	2.54	5	2.38	3
progc	38.7	2.67	2	2.54	5	2.44	3
progl	70.0	1.88	1	1.78	5	1.70	3
trans	91.5	1.63	2	1.53	5	1.46	4

Öğrenci bib dosyasını örnek verdi: IDBE 1,69 BPC’ye inerken diğerleri 2,11 ve 1,93’te kaldı. Farklı uzantılı dosyalarla yapılan ikinci bir karşılaştırmada da IDBE’nin hem BPC hem dönüşüm süresi bakımından daha iyi olduğu, yalnız BWT’ye göre daha düşük iletim süresi sağladığı belirtildi.

Ders sonrası not: Öğrenci IDBE’nin bütün dosyalarda daha iyi sonuç verdiğini söy-

ledi. Tabloya göre geo dosyasında IDBE (5,18) ikisinden de kötüdür, paper1'de ise *Encode (1,59) IDBE'den (2,26) iyidir. IDBE 14 dosyanın 12'sinde* Encode'dan iyidir. Ortalama BPC python ile hesaplandı: BWT 2,78, BWT + *Encode 2,52, BWT + IDBE 2,40. Dönüşüm süresinde IDBE* Encode'dan hızlı, ama yalnız BWT'den yavaştır.

6.6 Hocanın BWT benzetmesi

Hoca BWT'nin son haftadaki bir seminer sunumunda ayrıca anlatılacağını söyleyip sezgisini şöyle verdi: elinizde çok renkli bir resim varsa onu 256 renge indirince resim sadeleşir. BWT de kelimelerdeki karakterlerin yerini, arka arkaya aynı karakterler gelecek biçimde değiştirir; böylece sonraki aşamalar (MTF, RLE) daha iyi sıkıştırma performansı verir.

7. BERT Tabanlı Kısa Metin Sıkıştırma

7.1 BERT

Altıncı sunumu yapan öğrenci, kısa metinlerin sıkıştırılması için BERT tabanlı bir yöntemi anlatan Türkçe bir makaleyi sundu. Makale Emir Öztürk ve Altan Mesut'a aittir; öğrenciye göre 2021'de Avrupa Bilim ve Teknoloji Dergisi'nde yayımlanmıştır.

Soru: Kısa metinler deyince mesajlar mı kastediliyor; girdi birkaç cümleden mi oluşuyor? **Cevap:** Çalışmada gazete metinlerinden oluşan büyük korpuslar kullanılmıştır, ama sıkıştırılan metinler farklı uzunluk aralıklarındaki (örneğin 0–99 bayt) kısa parçalardır.

Öğrenci önce BERT'i tanıttı. Google'ın 2018'de duyurduğu BERT (Bidirectional Encoder Representations from Transformers), cümleyi iki yönlü (soldan sağa ve sağdan sola) değerlendirir; böylece anlam ve kelimeler arası ilişkiler daha iyi çıkarılır. Kitap korpusu ve Wikipedia ile BERT Large ve BERT Base modelleri eğitilmiştir; öğrenciye göre BERT Large 16, BERT Base 4 TPU ile 4 gün boyunca eğitilmiştir.

BERT iki teknikle eğitilir:

- Maskeli dil modeli (MLM): cümledeki kelimelerin %15'i seçilir. Seçilenlerin %80'i maske etiketiyle, %10'u rastgele bir kelimeyle değiştirilir, %10'u değiştirilmeden bırakılır. %15 oranı, çok fazla kelime maskelenirse tahminin zorlaşacağı düşünülerek seçilmiştir. Yalnız maskelenen kelimeler tahmin edilir.
- Sonraki cümle tahmini (NSP): ikinci cümlenin ilk cümlenin devamı olup olmadığı tahmin edilir. Önişlemede ikinci cümlelerin %50'si değiştirilir, %50'si olduğu gibi bırakılır.

7.2 Sıkıştırma aşamaları

Yöntem de BERT'in tahmin işlemine dayanır: bir maskeleme etiketi kullanılır, sonra açma aşaması gelir. Öğrencinin örneği: "bugün hava …" cümlesinde BERT boşluğa "güzel", "güneşli" gibi sık kullanılan kelimeleri önerebilir; bu öneriler bir tahmin listesi oluşturur. Önemli parametreler pencere boyutu (kaç kelimeye bakılacağı) ve tahmin edilecek kelime sayısıdır; makalede ikisi de değiştirilerek denenmiştir. Karışıklık olmasın diye genelde son kelimenin (cümle sonunun) tahmini seçilmiştir. İstenen tahmin sayısı her zaman dolmayabilir; BERT daha kısa bir liste döndürebilir.

Slayttaki sıkıştırma şeması (ekrandan, 01:38:35):

```
Cümle: K1 K2 K3 K4 K5 K6
Pencere (K1..K3) + maske -> BERT -> Tahmin listesi T1..Tn
Sıradaki kelime K4 tahmin listesinde:
  bulundu    -> listedeki indisi, tahmin indisleri dizisine (I1..In) eklenir
  bulunmadı  -> I dizisine 0 bayt değeri eklenir,
                K4 tahmin edilemeyen kelimeler dizisine (D1..Dn) eklenir
I dizisi -> FSE    \
                    -> Sıkıştırılmış veri
D dizisi -> Huffman /
```

Tahmin aşamasının sonunda iki dizi tek bir veri halinde kaydedilir.

7.3 Açma aşamaları

Slayttaki açma şeması (ekrandan, 01:40:00):

```
Sıkıştırılmış veri -> FSE açma -> I1..In ;  Huffman açma -> D1..Dn
Sıradaki indis 0 mı?
  evet  -> D dizisinden sıradaki kelime okunur, çıktıya yazılır
  hayır -> pencerenin (K(n-2), K(n-1), Kn) sonuna maske etiketi eklenir,
           BERT tahmin listesi T1..Tn üretilir,
           indisin gösterdiği kelime K(n+1) olarak çıktıya yazılır
Her adımda pencere güncellenir.
```

> **Ek bilgi (derste anlatılmadı):** FSE (Finite State Entropy), asimetrik sayı sistemleri (ANS) tabanlı bir entropi kodlayıcısıdır. Aritmetik kodlamaya yakın sıkıştırma oranını tablo tabanlı ve hızlı biçimde sağlar; Zstd'nin entropi kodlama aşamasında da kullanılır.

7.4 Karşılaştırılan yöntemler ve sonuçlar

Yöntem beş kayıpsız sıkıştırıcıyla karşılaştırıldı. Slayttaki açıklamalar (ekrandan, 01:42:40):

- Gzip: Phil Katz'ın geliştirdiği DEFLATE'e dayanır; tekrar eden yapıları LZSS ile azaltır, ardından Huffman uygular.
- Bzip2: girdi bloklara ayrılır ve sıralanır; ardından MTF ve benzeri dönüşümler, son aşamada Huffman yapılır.
- Zstd: DEFLATE'in oranlarına daha hızlı ulaşmayı amaçlar; LZ77 tabanlıdır ve daha büyük arama penceresi kullanır; entropi kodlamada FSE ve Huffman'ı birlikte kullanır.
- Smaz: kısa metinleri iyi oranlarda ve hızlı sıkıştırır; küçük harfli İngilizce metinlere özgü statik bir sözlük içerir; "http://" ve "</" gibi girdiler içerdiği için HTML ve URL'leri de biraz sıkıştırabilir.
- Shoco: İngilizce metinlerin sıkıştırılmasında entropi kodlayıcıdır, sözlük tabanlı değildir; genellikle Smaz'dan kötü oran verir. Metinde sayılar gibi sözlükte olmayan karakterler çoksa Smaz metni genişletebilir; Shoco ASCII metinlerde genişletme yapmaz.

Soru: Son iki yöntem (Smaz ve Shoco) nedir, hiç duymadım? **Cevap:** Öğrenci slayttaki kısa açıklamaları okudu. Shoco TR'nin çok başarılı olduğunu, bunun büyük olasılıkla Türkçe metin üzerinde eğitildiği için olduğunu söyledi. Hoca bu yöntemlerin dile bağımlı olduğunu, Smaz'ın İngilizceye özgü olduğunu ve HTML/URL gibi çok kalıp içeren metinlerde işe yarayabileceğini belirtti.

BERT yönteminde W pencere boyutu, P tahmin listesi uzunluğudur. Sabah veri seti için metin uzunluğuna göre sıkıştırma oranları (%; küçük değer iyi), seçilmiş satırlar (ekrandan, 01:42:40):

Metin boyutu	Gzip	Bzip2	Zstd	Smaz	Shoco	ShocoTR	BERT en iyi
0-99	165,4	228,2	134,8	131,0	128,9	122,2	102,8
100-199	104,1	114,3	95,0	104,8	108,6	72,5	54,4
500-599	71,0	74,7	72,1	103,1	108,3	71,3	45,2
900-	60,4	62,0	61,9	104,0	108,9	71,4	41,7

Çok kısa metinlerde genel sıkıştırıcılar veriyi büyütür (oran %100'ün üzerinde). İngilizceye odaklı Smaz ve Shoco Türkçe metinde %100'ün üzerinde kalır, yani veriyi genişletir. Türkçe veriyle eğitilen BERT modeli ve Shoco TR en iyi sonuçları verir; öğrenciye göre BERT özellikle 500 bayttan küçük metinlerde avantajlıdır. Sabah, Cumhuriyet ve bir üçüncü veri setiyle de sonuçlar verilmiştir.

8. Kayıplı Görüntü Sıkıştırmada Öğrenilmiş Yinelemeli Kod Çözücü

8.1 Amaç

Yedinci sunumu yapan öğrenci, 2019 tarihli "Learned Neural Iterative Decoding for Lossy Image Compression Systems" makalesini anlattı. Makale, görsel sıkıştırmada kod çözücünün yeniden yapılandırma sürecini iyileştirmek için "yinelemeli iyileştirme" (iterative refinement)

adlı bir algoritma önerir. RNN yapısı ve kendi kendine bağlı bellek birimleri kullanılarak pikseller arasındaki nedensel ve nedensel olmayan bilgilerden yararlanılır. Kodak kayıpsız gerçek renkli görüntü kümesi üzerinde JPEG 2000 ve bazı sinir ağı modellerine göre daha düşük bozulma (distortion) elde edilmiştir.

Uçtan uca sinir ağı sistemlerinde etkili kodlama ve kod çözme fonksiyonları gerekir. Bazı yapıların ayrık ve değişmez nicemleyiciler içermesi geri yayılımla öğrenmeyi zorlaştırır. Birçok görsel sıkıştırma tekniği yalnız hedef bloğun değerlerini kullanır, çevredeki blokların bağımlılıklarını hesaba katmaz. Bu yöntem çevre blokların istatistiksel bağımlılıklarını da kullanır. Makalenin yanıtladığı soru şudur: bir kodlayıcı ve nicemleme şeması verilirse, en iyi kod çözücü nasıl olur?

8.2 İlgili çalışmalar

Slayttaki ilgili çalışmalar (ekrandan, 01:50:00):

- JPEG: 8×8 bloklara ayrık kosinüs dönüşümü (DCT), ardından frekans katsayılarının seyreklik örüntüsünü elde etmek için run-length kodlama.
- JPEG 2000: görüntüyü herhangi bir boyutta (örneğin 64×64) parçalara ayırır, çok ölçekli ayrık dalgacık dönüşümü (DWT) uygular, ardından uyarlamalı ikili aritmetik kodlayıcı ve bit akışı düzenlemesi yapar.
- DCT ve DWT evrenseldir ama belirli piksel dağılımlarından yararlanamaz; istatistiksel öğrenme teknikleri daha güçlüdür.
- Van den Oord, görüntü piksellerini iki uzamsal boyut boyunca tahmin eden bir sinir ağı önermiştir.
- Toderici, evrişimli ve ters evrişimli LSTM modellerine dayanan değişken oranlı görüntü sıkıştırma çerçevesi önermiştir. Öğrenciye göre bu, Kodak görüntüleriyle JPEG'i geçen ilk derin ağ çalışmalarındandır ve bu makale onun üzerine kurulmuştur.

Makale kendini iki yönden ayırır; biri, bir dönüşüm yerine RNN tabanlı bir tahminci kullanmasıdır.

8.3 Yinelemeli iyileştirme

Görüntü $d \times d$ boyutlu bloklara ayrılır ve düzleştirilir. Üç fonksiyon kullanılır:

- Dönüşüm fonksiyonu: o anki blok hesaplanırken komşu blokların değerlerinin bir özetini çıkarır.
- Durum fonksiyonu: bu özeti kullanarak bir filtreleme yapar.
- Yeniden yapılandırma (reconstruction) fonksiyonu: bunları kullanarak hedef bloğu kurar.

Slayttaki şekil (ekrandan, 01:53:10): soldaki klasik zincirde görüntü dönüşüm (DCT, DWT), nicemleyici, Huffman kodlama, Huffman kod çözme, ters nicemleyici ve ters dönüşümden geçer. Önerilen sistemde Huffman kod çözmeden sonra ters nicemleme ve ters dönüşüm yerine bir sinirsel kod çözücü gelir. Kod çözücü bir Elman RNN tahmincisidir; şekil 2 bölüm (episode) ve bölüm başına $K = 3$ adım gösterir. Kesikli oklarla her blok hesaplanırken komşu bloğun bilgisi bir sonraki adıma aktarılır; böylece komşuluktan gelen istatistikler kullanılır. Geri yayılım yalnız bir bölüm içinde uygulanır; bölüm sonunda oluşan bellek bir sonraki bölüme taşınır.

Algoritmaya görüntü, kod çözücü parametreleri ve adım sayısı verilir. Önce her bloğun nicemlenmiş değerleri hesaplanır, sonra komşuluk değerleri, ardından her blok içindeki değerler hesaplanır. Dönüşüm kısmında çok katmanlı algılayıcı kullanılır.

8.4 Kayıp fonksiyonu

Amaç bozulma D'yi en iyi duruma getirmektir. Yalnız ortalama karesel hata (MSE) ile eğitildiğinde JPEG'e göre PSNR kazancı yetersiz kalmıştır. Bu yüzden MAE ve MSE'nin dışbükey birleşimi olan bir kayıp tanımlanmıştır (ekrandan, 01:56:10):

$$\mathcal{D}(\mathbf{p}^j, \hat{\mathbf{p}}^j) = (1-\alpha)\,\mathcal{D}_{MAE}(\mathbf{p}^j, \hat{\mathbf{p}}^j) + \alpha\,\mathcal{D}_{MSE}(\mathbf{p}^j, \hat{\mathbf{p}}^j)$$

α iki bozulma terimi arasındaki dengeyi ayarlar; denemelerde $\alpha = 0{,}235$ en iyi sonucu vermiştir. MAE en küçük mutlak sapmaları, MSE ise bütün piksellerin karesel sapmasını hesaba katar; α ikisi arasında denge kurar. Kod çözücü parametreleri, K adımlı yeniden yapılandırma bölümleri üzerinde, B hedef görüntü parçasından oluşan mini-batch'lerle optimize edilir.

Soru: K ve B neydi; pencerenin, yani piksel bloğunun boyutları mı? **Cevap:** Öğrenci "evet" dedi.

> **Ders sonrası not:** Slayta göre K büyük olasılıkla bir bölümdeki yeniden yapılandırma adımı sayısıdır (şekilde $K = 3$), B de mini-batch'teki hedef görüntü parçası sayısıdır. Blok boyutu ayrıca $d \times d$ ile gösterilir.

8.5 Deneyler

Places365 veri setinden 512×512 boyutunda 128.000 görüntü ve bir RAW görüntü veri setinden yaklaşık 7.000 görüntü alınmış; JPEG ve JPEG 2000 ile belirli bit/piksel aralığında sıkıştırılmıştır. Tek bir görüntü kanalına odaklanıldığı için yalnız gri tonlu görüntülerle çalışılmış, ama yöntemin RGB gibi biçimlerde de çalışabileceği belirtilmiştir. JPEG'de 8×8, JPEG 2000'de 64×64 bloklar kullanılmış; test için Kodak dahil 6 veri seti kullanılmıştır.

Model tek katmanda 512 gizli birim kullanır; parametreler düzgün (uniform) dağılımla ilklendirilmiştir. İki öğrenme oranı takvimi denenmiştir: biri 0,002'den başlayıp sabit biçimde değişen, diğeri önceki çıktılara göre değişen. İkinci takvim yakınsamayı hızlandırmıştır. Slayta göre sahte korelasyonları önlemek için iki adımlı bir karıştırma kullanılmıştır (ekrandan, 02:00:00):

- Her epoch başında görüntüler rastgele karıştırılır.
- Epoch içinde her görüntü için tarama başlangıç köşesi rastgele seçilir: sol üst, sol alt, sağ üst ya da sağ alt.

Sonuçta önerilen yöntem karşılaştırılan diğer yöntemlere göre aynı sıkıştırma oranında daha düşük bozulma vermiştir. Adım sayısı K artırıldıkça daha iyi sonuç alınmıştır. Hoca çalışmayı oldukça karışık bulduğunu, ama iyi toparlandığını söyledi.

9. RNN ile Tam Çözünürlüklü Görüntü Sıkıştırma

Son sunumu yapan öğrenci, derin öğrenmeyle görüntü sıkıştırma üzerine bir makaleyi İngilizce anlattı. Bu bölümün altyazısı büyük ölçüde bozuktur; aşağıdaki bilgiler anlaşılabilen cümlelerden ve slaytlardan alınmıştır.

9.1 Problem

Sinir ağlarıyla görüntü sıkıştırmada, ağ genellikle darboğaz (bottleneck) katmanının boyutuna bağlı sabit bir sıkıştırma oranı kullanır. Farklı oranlar için ağı yeniden eğitmek zordur. Bu çalışma, kullanım sırasında ağı yeniden eğitmeden değişken sıkıştırma oranı sağlar. Ağ, kodlama adımları, ikilileştirme (binarization) ve kod çözme adımlarından oluşur ve yinelemeli çalışır.

9.2 Klasik kodlama zinciri

Öğrenci kodlamanın genel adımlarını slaytla gösterdi (ekrandan, 02:10:00):

```
Kaynak görüntü -> Kaynak kodlayıcı -> Nicemleyici -> Entropi kodlayıcı -> Sıkıştırılmış sinyal
- Kaynak görüntü: genellikle küçük bloklara ayrılır (JPEG 8x8 piksel blok kullanır).
- Kaynak kodlayıcı: sinyali frekans alanına taşır (DFT, DCT, DWT). Bu adım kayıpsızdır.
- Nicemleyici: yüksek frekans bileşenlerini atar. Bu adım kayıplıdır.
- Entropi kodlayıcı: her sembolün olasılığı bilgisine dayanarak sinyali mümkün olan
  en az bitle gösterir; en yaygın model Huffman kodlamasıdır. Bu adım kayıpsızdır.
- Sıkıştırılmış sinyal: 0101011001010...
```

9.3 Yinelemeli birimler

Ağda LSTM, ilişkisel (associative) LSTM ve GRU türü yinelemeli birimler denenmiştir. LSTM formüllerinde c_t hücre durumu, h_t gizli durum, t yineleme indisidir; önceki hücre durumu c_{t-1},

önceki gizli durum h_{t-1} ile gösterilir. İlişkisel LSTM, LSTM'i holografik indirgenmiş gösterimler (HRR) fikriyle birleştirerek verinin anahtar–değer biçiminde saklanmasını sağlar; HRR iki vektör (anahtar ve ilişkili içerik) arasında bir bağlama işlemi kullanır. İlişkisel LSTM yalnız kod çözücüde kullanıldığında etkili olmuştur. GRU, LSTM'den daha az parametreye sahiptir; evrişimli sürümünde çarpma yerine evrişim kullanılır.

9.4 Entropi kodlama

Entropi kodlama, ortamın özelliklerinden bağımsız bir kayıpsız sıkıştırma şemasıdır: her sabit uzunluklu girdi sembolünü karşılık gelen değişken uzunluklu önek kod kelimesiyle değiştirir. Makaledeki aşamalı (progressive) entropi kodlayıcıda, k'inci yinelemenin satır LSTM'ine verilen veri, önceki yinelemelerden gelen bilgiyle zenginleştirilir. Slayttaki ağ (ekrandan, 02:20:00):

```
Önceki yinelemenin kodları -> 3x3 Conv -> 3x3 Conv -> 1x1 Conv LSTM -> 1x1 Conv -> 1x1 Conv ->
(ara katmanlar 64 kanallı)
```

9.5 Sonuçlar

Değerlendirme ölçütleri çok ölçekli yapısal benzerlik (MS-SSIM) ve insan görsel sistemine göre PSNR (PSNR-HVS) olmuştur; grafiklerde yatay eksen bit/piksel, dikey eksen PSNR'dir. Dört birim (GRU, artık GRU, LSTM, ilişkisel LSTM) ile üç yeniden yapılandırma yöntemi (tek atış, toplamalı, ölçeklenmiş artık toplamalı) birlikte denenmiş; bir milyon eğitim adımı yapılmıştır. 32×32 eğitim verisiyle her iki ölçütte de en iyi sonucu GRU (tek atış) vermiştir (ekrandan, 02:30:00). Görsel karşılaştırmada önerilen yöntemin düşük bit/piksel değerlerinde rengi daha iyi koruduğu ve nesnelerin çevresinde gürültü oluşturmadığı gösterilmiştir. Öğrenci sonuçları iyileştirmek için veri artırma, daha çok eğitim görüntüsü ve farklı algoritma birleşimlerinin denenmesini önerdi.

> **Ek bilgi (derste anlatılmadı):** Bu sunumdaki çalışma büyük olasılıkla Toderici ve arkadaşlarının "Full Resolution Image Compression with Recurrent Neural Networks" (2017) makalesidir. MS-SSIM görüntüleri birden fazla ölçekte yapısal benzerlikle karşılaştırır; PSNR-HVS ise PSNR'yi insan gözünün frekans duyarlılığına göre ağırlıklandırır. İkisinde de büyük değer daha iyi kalite demektir.

Örnek Sorular

Soru 1 (ekrandan, 00:43:40): Dinamik v2vDC'nin ayrıştırma algoritmasında, yeni her kelime için bir terminal sembol ve (ikinci kelimeden başlayarak) önceki terminal sembolle birleşik bir non-terminal sembol üretilir. "the more I know about you" metni için oluşan sembolleri sırasıyla yazın.

> **Çözüm:** S_0 = the, S_1 = more, S_2 = the more, S_3 = I, S_4 = more I, S_5 = know, S_6 = I know, S_7 = about, S_8 = know about, S_9 = you, S_{10} = about you. Toplam 11 sembol vardır; non-terminal semboller (2, 4, 6, 8, 10) sol ve sağ bileşenlerinin indisleriyle saklanır: (0,1), (1,3), (3,5), (5,7), (7,9).

Soru 2 (derste sözlü, 00:40:33): v2vDC ile sıkıştırılmış veri üzerinde arama yapılabilir mi?

> **Çözüm:** Evet. Kelimeler ve kelime dizileri kod kelimeleriyle değiştirildiğinden, aranan kelimenin kodu bulunup sıkıştırılmış metinde doğrudan aranabilir.

Soru 3 (derste sözlü, 00:49:03): Dinamik v2vDC'de gönderici iki sembolden sonra neden boşluk bırakır?

> **Çözüm:** Alıcıya yeni bir birleşik (non-terminal) sembol eklendiğini bildirmek için. Alıcı aynı birleşimi kendi tarafında yaparak göndericiyle eşzamanlı kalır.

Soru 4 (ekrandan, 00:18:40): FP32'den INT8'e nicemlemede ölçek çarpanı ve nicemleme fonksiyonu nasıl tanımlanır?

> **Çözüm:** scale $= \dfrac{f_{max} - f_{min}}{q_{max} - q_{min}}$, $Q(x) = \lfloor x/\text{scale} + \text{zero point} \rfloor$. f_{min}, f_{max} girdi verisinin, q_{min}, q_{max} nicemlenmiş verinin sınırlarıdır.

Soru 5 (derste sözlü, 01:22:42): IDBE'nin sözlüğü harfi eksik kelimelerden mi oluşur?

> **Çözüm:** Hayır. Eksik harf örneği yalnız "önceden bilinen bir sözlükle dönüşüm" fikrini anlatmak içindir. Algoritma token'ı tabloda birebir arar; bulursa "uzunluk işaretçisi (ASCII 251–254) + kod" yazar, bulamazsa token'ı olduğu gibi yazar. En yakın eşleşme aranmaz.

Soru 6 (ekrandan, 01:23:40): IDBE çıktısında metnin kendisinde ASCII 251–255 arasındaki bir karakter geçerse ne yapılır?

> **Çözüm:** Karakter bir kez daha yazılır; böylece kod çözücü bunun bir işaretçi değil, metnin parçası olduğunu anlar.

Soru 7 (derste sözlü, 01:41:02): Smaz ve Shoco nedir, kısa Türkçe metinlerde neden zayıf kalırlar?

> **Çözüm:** Smaz küçük harfli İngilizce metinlere, HTML ve URL'lere yönelik statik sözlük kullanır; Shoco İngilizce metinler için entropi kodlayıcıdır. İkisi de dile bağımlı olduğundan Türkçe metinde oranları %100'ün üzerine çıkar, yani veriyi büyütür; Türkçe için uyarlanan Shoco TR ise daha başarılıdır (ekrandan, 01:42:40).

Soru 8 (ekrandan, 01:56:10): Öğrenilmiş yinelemeli kod çözücüde kullanılan kayıp fonksiyonunu yazın ve α'nın rolünü açıklayın.

> **Çözüm:** $\mathcal{D} = (1-\alpha)\mathcal{D}_{MAE} + \alpha\mathcal{D}_{MSE}$. α, mutlak hata ile karesel hata arasındaki dengeyi ayarlar; makalede $\alpha = 0{,}235$ en iyi değer bulunmuştur. Yalnız MSE ile eğitim JPEG'e göre yetersiz PSNR kazancı vermiştir.

Dersin Toparlaması

Bu ders sekiz seminer sunumuyla sinir ağlarıyla sıkıştırmanın iki yüzünü gösterdi. Birincisi, büyük sinir ağı modellerinin kendisinin sıkıştırılmasıdır: CNN'lerde budama, nicemleme ve Huffman birlikte kullanılarak doğruluk korunurken model onlarca kat küçültülür; FastText.zip'te ürün nicemleme, budama ve hashing ile metin sınıflandırma modelleri birkaç on kilobayta indirilir. İkincisi, sinir ağlarının sıkıştırmada tahminci olarak kullanılmasıdır: DeepZip RNN tahminleriyle aritmetik kodlayıcıyı besler, BERT tabanlı yöntem kelimenin yerine tahmin listesindeki indisini yazar, öğrenilmiş kod çözücüler ise JPEG ve JPEG 2000 çıktısını komşu blok bilgisiyle daha iyi yeniden kurar.

Klasik sözlük yöntemleri de ele alındı: dinamik v2vDC kelime dizilerinden eşzamanlı güncellenen bir sözlük kurar ve sıkıştırılmış metinde aramaya izin verir; IDBE önceden hazırlanmış bir sözlükle metni BWT'ye daha uygun bir ara forma dönüştürür. Hoca BWT'nin son haftadaki bir seminerde ayrıca anlatılacağını belirterek sezgisini, bir resmi 256 renge indirip sadeleştirmeye benzetti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.